CCSAR

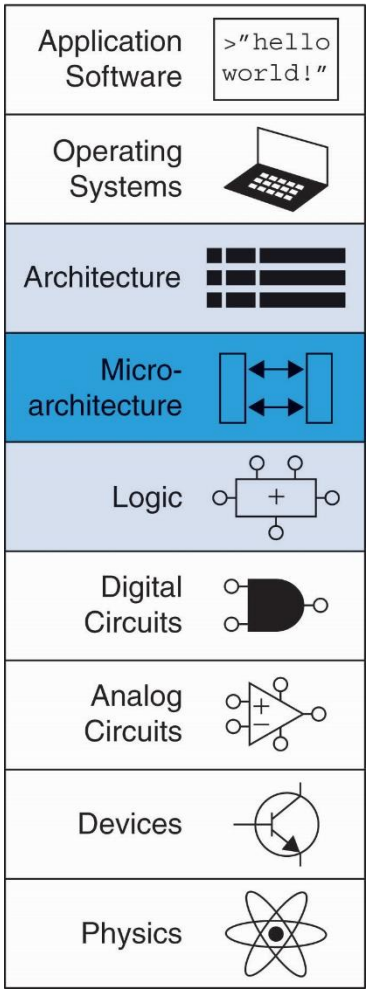
The Processor: Single-cycle Implementation

Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Introduction

- **A particular architecture can be implemented by one of many different microarchitectures with different price/performance/power trade-offs**
  - E.g., the Intel Core i7, the Intel 80486, and the AMD Athlon all implement the x86 architecture with different microarchitectures
- **We divide microarchitectures into two interacting parts**
  - **Datapath** – functional blocks
  - **Control** – control signals



Introduction

- **Datapath**
  - Contains the **hardware** necessary to perform operations required by the processor
  - Reacts to what the controller tells it
    - E.g., “I was told to do an add, so I’ll feed these arguments through an adder”

- **Control**
  - **Decides** what each piece of the datapath should do
    - E.g., What operation am I performing? Do I need to get info from memory? Should I write to a register? Which register?
  - Must make decisions based on the input instruction only

Introduction

- **Multiple implementations for a single architecture**
  - **Single-cycle**: Each instruction executes in a single cycle
  - **Multicycle**: Each instruction is broken up into series of shorter steps
  - **Pipelined**: Each instruction broken up into series of steps and multiple instructions execute at once
- **Challenge is to satisfy constraints of cost, power, and performance**

Introduction

- **CPU performance factors**
  - **Instruction count**: Determined by ISA and compiler
  - **CPI and Cycle time**: Determined by CPU hardware

Execution Time = (Instruction Count) * (Cycles Per Instruction) * (Time Per Cycle)

RISC-V Implementations

- **We will examine two RISC-V implementations**
  - A simplified single-cycled implementation (**this lecture**)
  - A pipelined implementation (next lectures)

- **Focus on a simple subset**
  - Memory reference: `lw`, `sw`
  - Arithmetic/logical: `add`, `sub`, `and`, `or`
  - Control transfer: `beq`

- **These instructions were chosen because they are sufficient to write useful programs**
  - Once you understand how to implement these instructions, you can expand the hardware to handle others

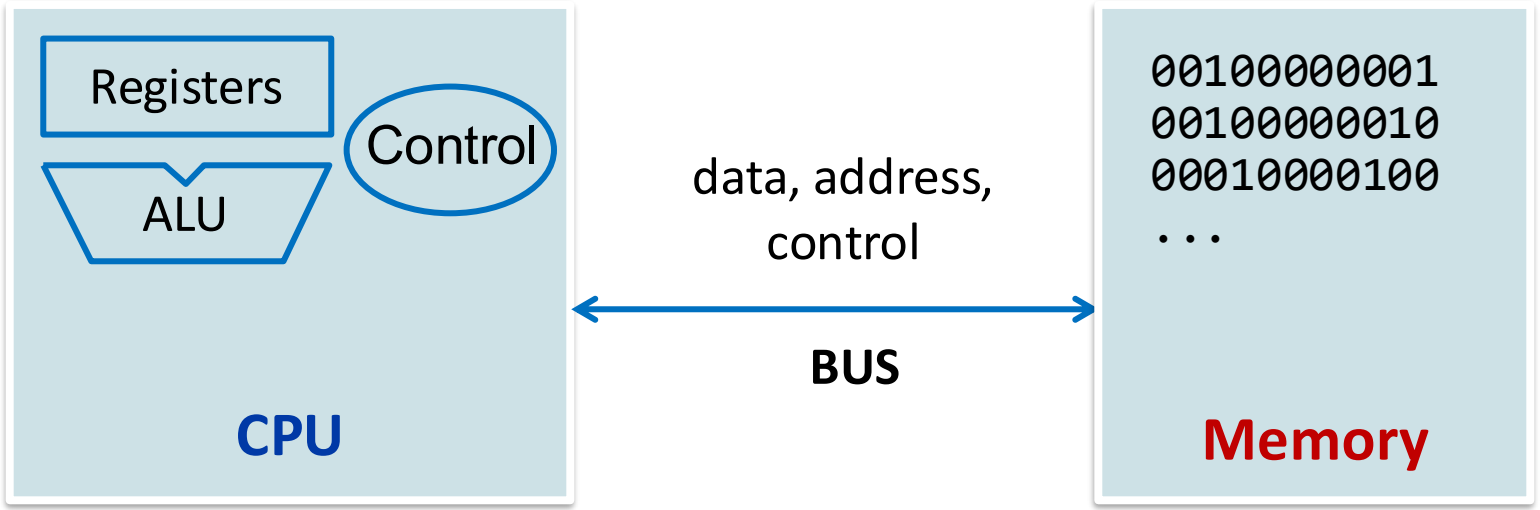
RISC-V Implementations

- **Most concepts used to implement this subset are used to construct a broad spectrum of computers**
  - From high-performance servers to general-purpose microprocessors to embedded processors

- **In examining the implementation, we will have the opportunity to see:**
  1. How the ISA determines many **aspects of the implementation**
  2. How the choice of various implementation strategies **affects the clock rate and CPI**
  3. How many of the **key design principles**, such as “Simplicity favours regularity”, can be illustrated

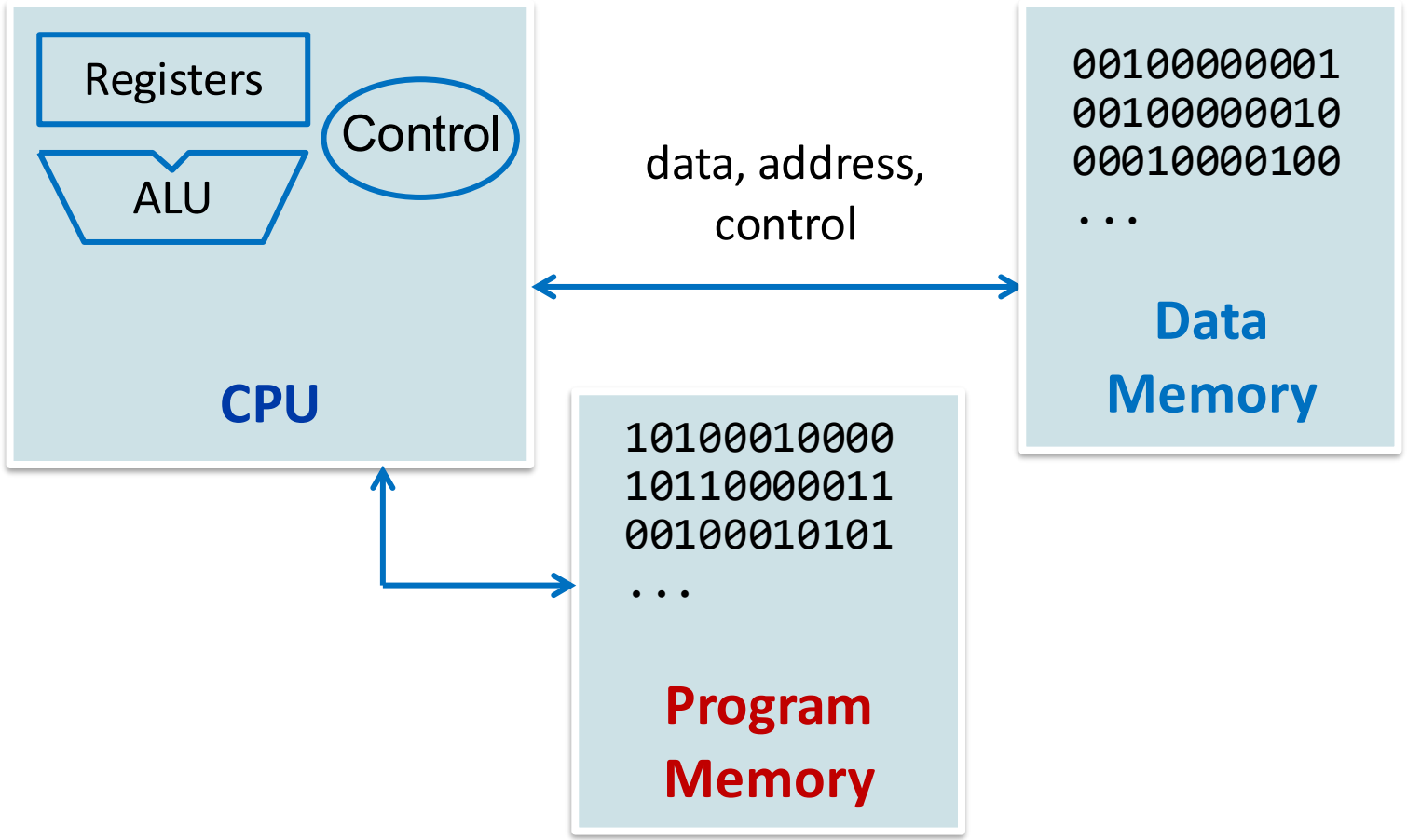
Basic Computer System

- **A processor executes instructions**
  - CPU has some internal state in storage elements (registers)
- **A memory holds instructions and data**
  - von Neumann architecture: combined instructions and data
  - Harvard architecture: separate instructions and data
- **A bus connects the two**



Basic RISC-V Processor

- **A RISC-V CPU with a (modified) Harvard architecture**
  - Modified: instructions and data in common address space, separate instructions/data caches can be accessed in parallel



Logic Design Conventions

- **To discuss the design of a computer, we must decide:**
  1. How the hardware logic implementing the computer will operate
  2. How the computer is clocked

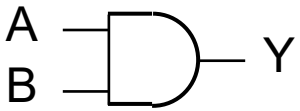
- **The datapath elements in the implementation consist of two different types of logic elements:**
  1. Elements that operate on data values (**combinational**)
  2. Elements that store values (**state**)

Building a Datapath

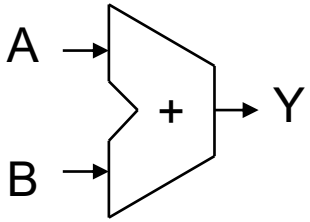
- **Information encoded in binary**
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- **Combinational element**
  - **Operate on data**
  - Output is a function of input
- **State elements**
  - **Store information**
  - Output can depend on both the inputs and the value stored in memory, which is called the state of the block

Combinational Elements

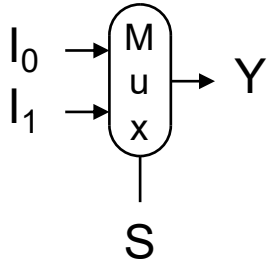
- **AND-gate**
  - Y = A & B



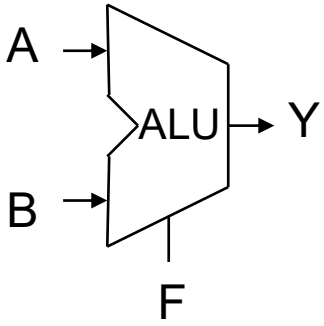
- **Adder**
  - Y = A + B



- **Multiplexer**
  - Y = S ? I_1 : I_0



- **Arithmetic/Logic Unit**
  - Y = F(A, B)



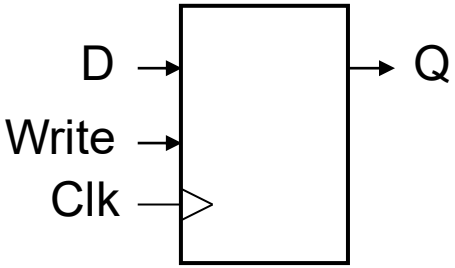
State Elements

- **Other elements in the design are not combinational, but instead contain state**
  - An element contains state if it has some internal storage
- **The architectural state for the RISC-V processor consists of the PC and the 32 32-bit registers**
  - While the overall system state also includes memory
- **Any RISC-V microarchitecture must contain this state**
  - Based on the current architectural state, the CPU executes a particular instruction with a particular set of data to produce a new architectural state

State Elements

- **State elements use a clock signal to determine when to update the stored value**
  - A clock is simply a free-running signal with a fixed cycle time (or clock period)

- **Have at least two inputs and one output**
  - The required inputs are the **data value** to be written into the element and the **clock**
  - The state element is **changed** only when the **write control signal is asserted**

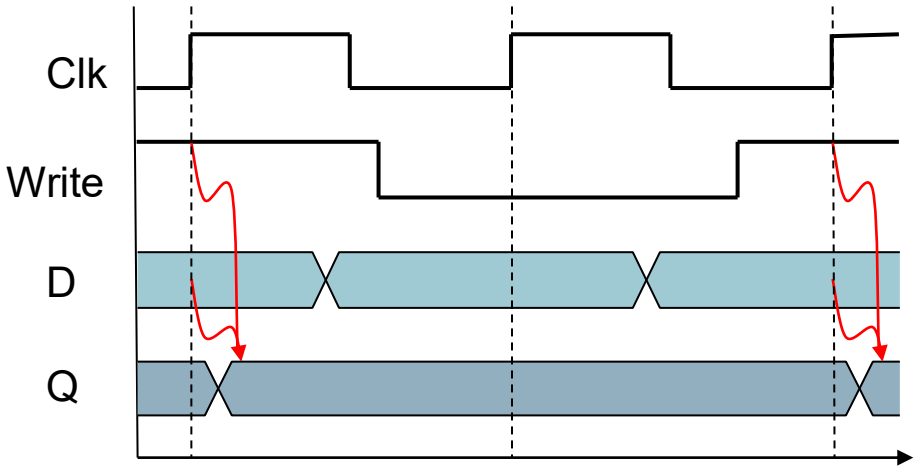


Clocking Methodology

- **A clocking methodology defines how and when data moves and state changes occur in a digital system**
- **In hardware, the clock acts like a metronome that coordinates all components**
  - Each “tick” of the clock tells the system when it’s safe to read signals, perform computations, and store results
- **This regular timing makes hardware predictable and synchronous**
  - Every component knows exactly when to update

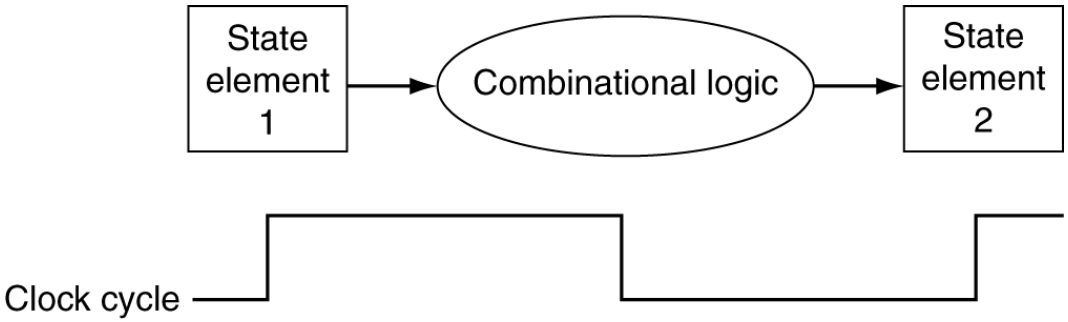
Clocking Methodology

- **In an edge-triggered design (used in most synchronous circuits):**
  - State changes happen **only at the rising or falling edge** of the clock signal — not in between
  - This means registers (flip-flops) capture their inputs at that clock edge and hold the value until the next edge
  - Between edges, combinational logic computes new results based on the current register outputs



Clocking Methodology

- **So, in each clock cycle:**
  1. Registers output their current stored values
  2. Those values pass through combinational logic (e.g., adders, multiplexers)
  3. At the next clock edge, the results are written back into registers



Clocking Methodology

- **The clock must have a long enough period so that all the signals in the combinational logic block stabilize**
  - This ensures that the next state component (e.g., flip-flop) captures the correct, stable output from the combinational logic
- **This constraint sets a lower bound on the length of the clock period**
  - Which must be long enough for all state element inputs to be valid
- **In our implementation, state elements are positive edge-triggered**
  - They update their stored values on the rising edge of the clock

Key takeaways

- **Two types of logic elements**
  - Combinational logic computes results from its inputs; state elements store values
- **State changes on the rising clock edge**
  - In our implementation, state elements are positive edge-triggered and update their stored values on the rising edge
- **Computation happens between clock edges**
  - Signals propagate through combinational logic and must stabilize before the next rising edge
- **Design implication**
  - The clock period must be long enough for the slowest combinational path to produce a valid result

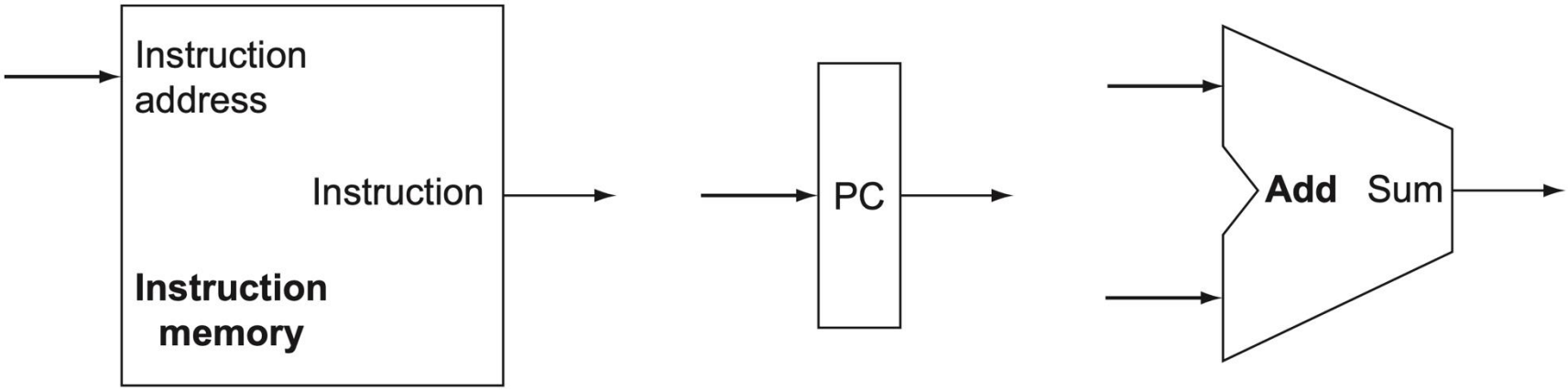
Fetch-Execute Cycle

- **Sequence of steps that the CPU follows to retrieve, interpret, and execute instructions stored in memory**
  - This cycle is fundamental to the functioning of any digital computer
- **This cycle continues from the moment a computer is switched on until it is turned off**
  1. **Fetching** is the initial stage where the CPU retrieves an instruction from the memory
  2. **Decoding** involves interpreting the fetched instruction
  3. **Executing** is the process of carrying out the decoded instruction
- **A single-cycle CPU fetches, decodes, and executes one instruction at a time**

Instruction Fetch

- **To execute any instruction, we must start by fetching the instruction from memory**
- **At the end of each clock cycle, the PC must be updated for the next instruction**
  - Normally, the PC advances to the next instruction (PC + 4)
  - Control-flow instructions may select a different next PC
- **Fetching instructions thus requires:**
  - **Memory unit** where instructions are stored
  - **Program counter** (PC) to hold the address of the current/next instruction
  - **Adder** to increment the PC to the address of the next instruction

Instruction Fetch

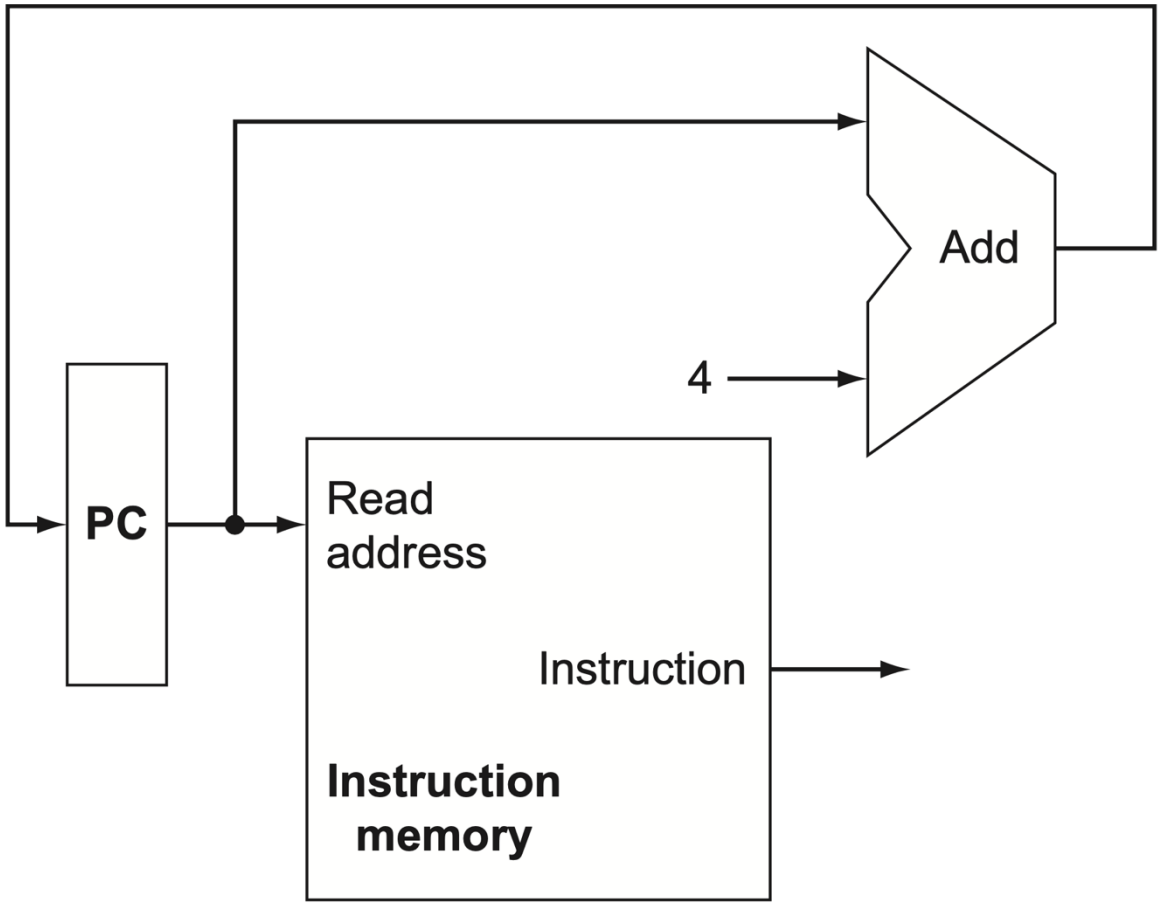


a. Instruction memory

b. Program counter

c. Adder

Instruction Fetch

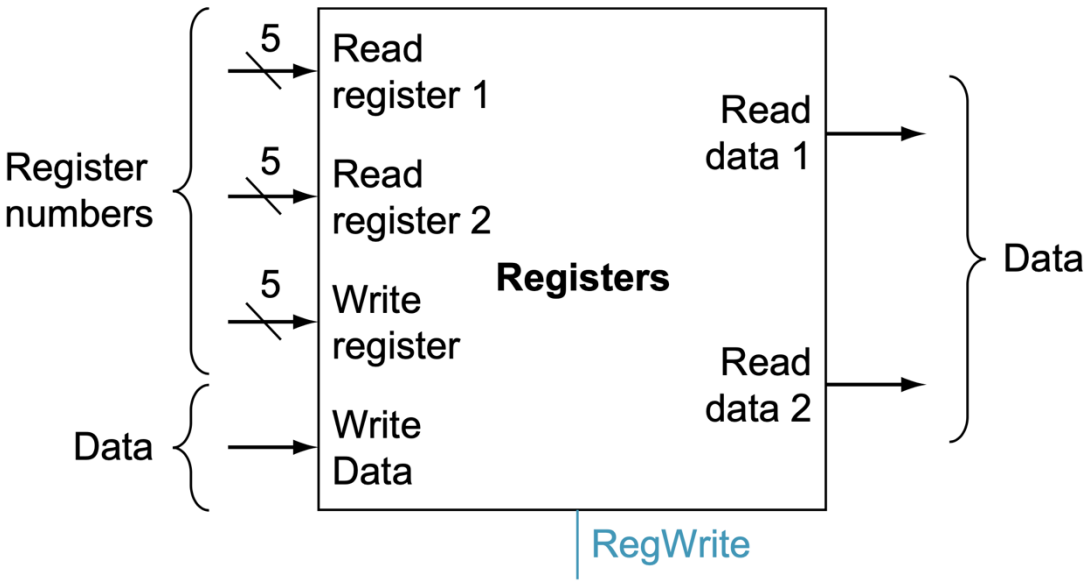


R-format Instructions

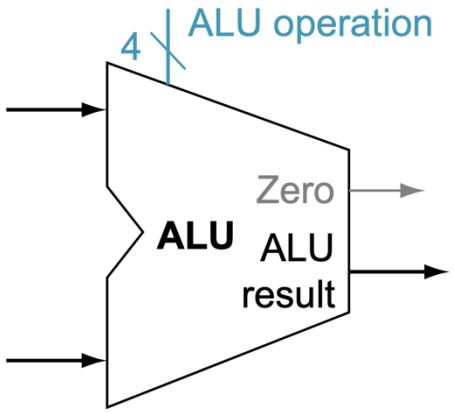
- **R-format instructions perform arithmetic-logical operations**
  - E.g., `add x1, x2, x3`
  - Read two registers
  - Perform an ALU operation on the contents of the registers
  - Write the result to a register

- **R-format instructions thus require:**
  - **Register file** where the register's contents are stored
  - **ALU** to operate on the values read from the registers

R-format Instructions



a. Registers



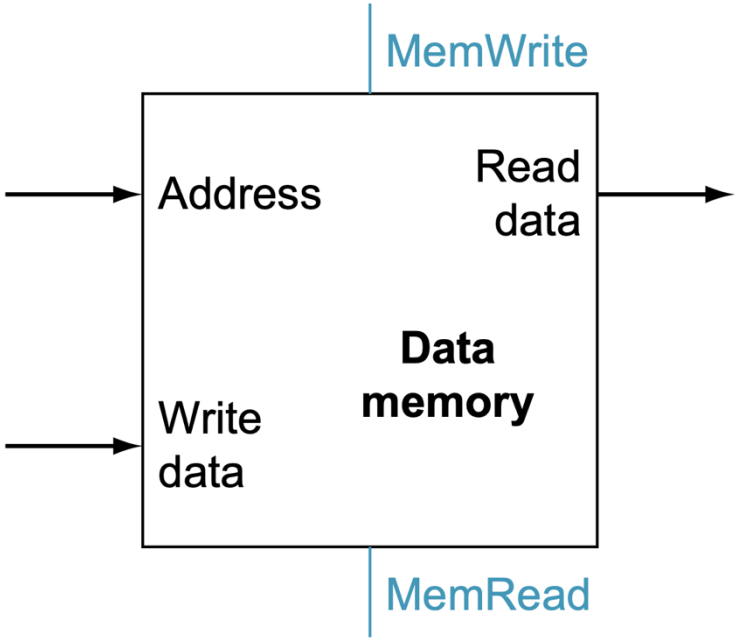
b. ALU

Load/Store Instructions

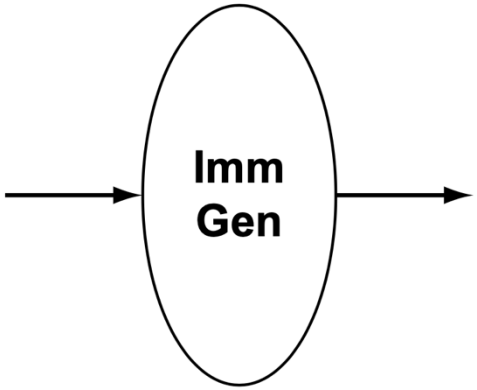
- **Memory reference instructions**
  - E.g., `lw x1, offset(x2)`
  - Compute a memory address
    - By adding a base register to a 12-bit signed offset field contained in the instruction
  - Perform a load or store operation

- **Memory reference instructions thus require:**
  - **Register file** where the register's contents are loaded/stored
  - **ALU** to operate on the value read from the base register and the offset
  - **Memory unit** where data is loaded/stored
  - **Immediate generation unit** to sign-extend the 12-bit offset field to a 32-bit signed value

Load/Store Instructions



a. Data memory unit



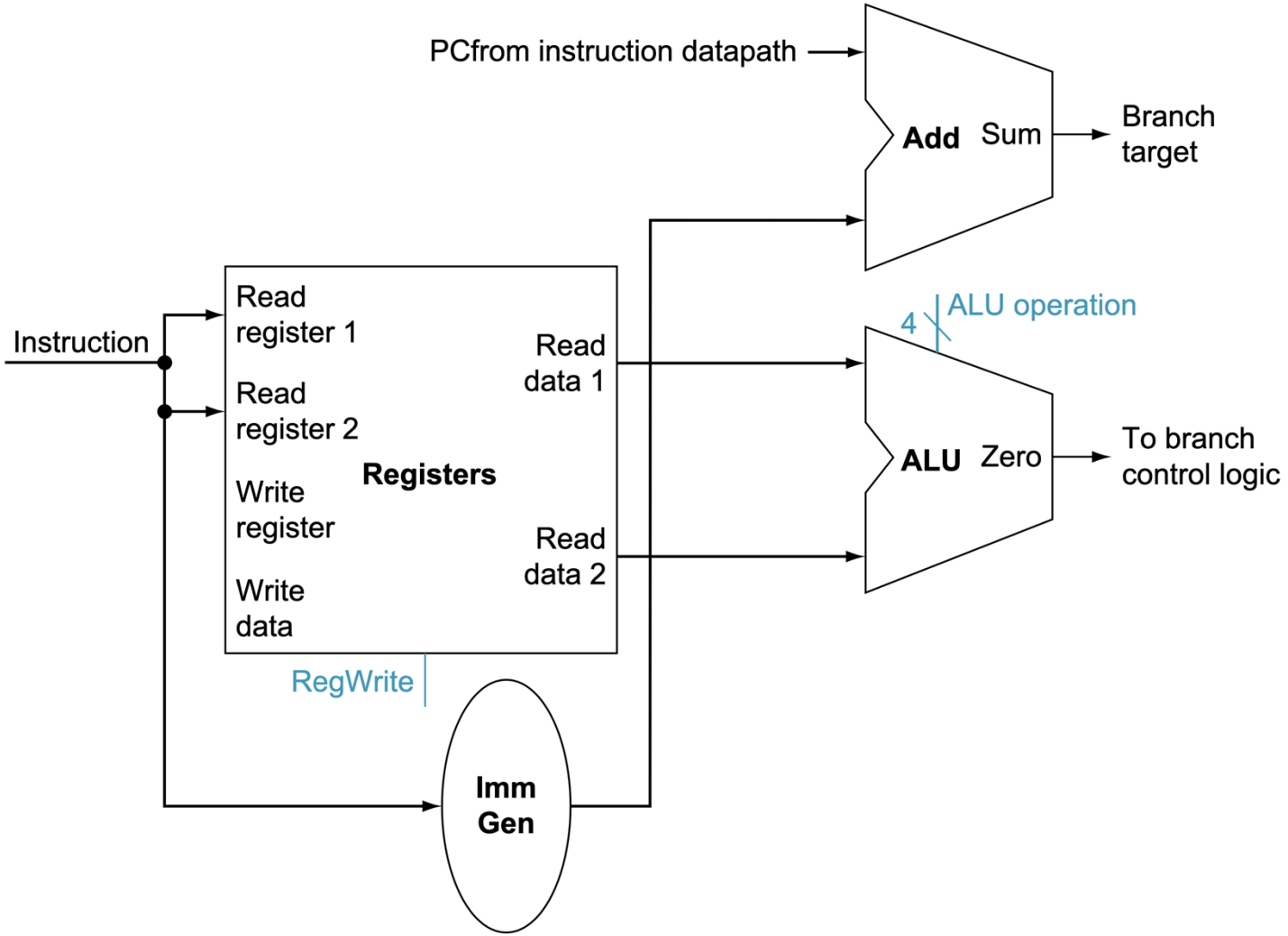
b. Immediate generation unit

Branch Instructions

- **Branch instructions**
  - E.g., `beq x1, x2, offset`
  - Compare two registers
  - Compute a branch target address:
    - By adding the PC to the sign-extended branch immediate
    - The encoded immediate represents an offset in multiples of 2 bytes
    - The least-significant bit of the branch offset is implicitly zero

- **Branch instructions thus require:**
  - **Register file** where the register's contents are stored
  - **ALU** to compare the values read from the registers
  - **Adder** to compute the branch target address

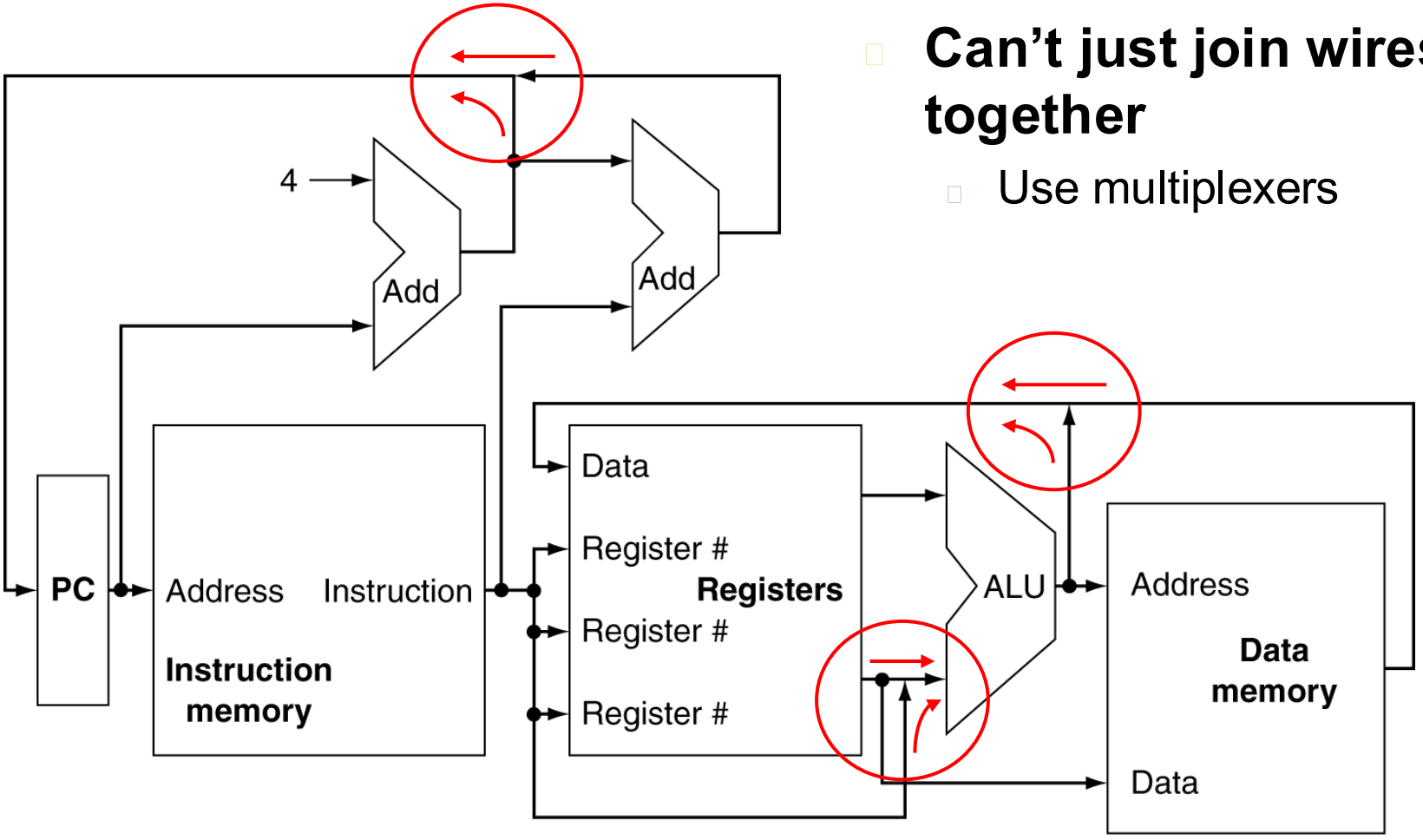
Branch Instructions



Combining the Elements

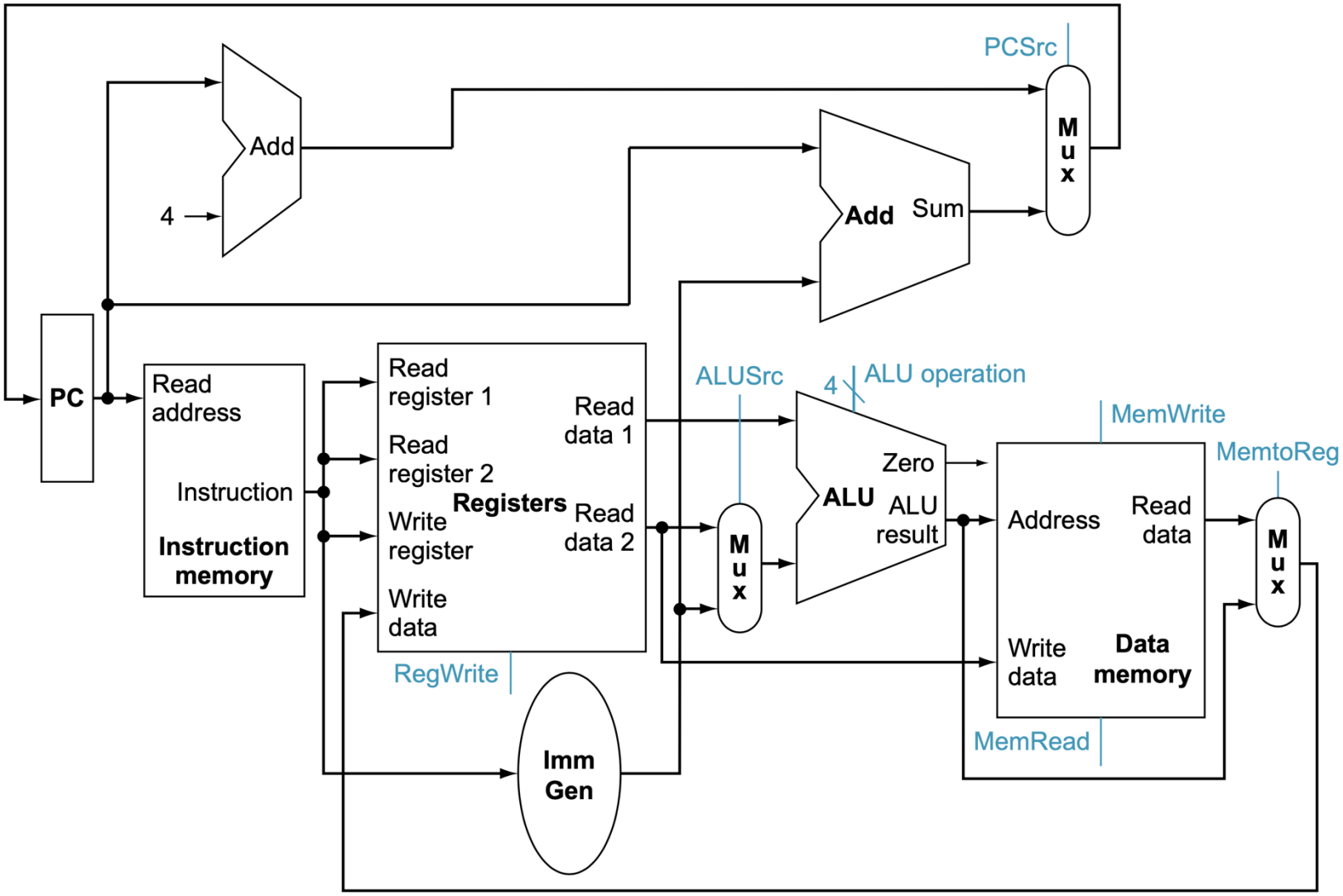
- **Our simple datapath executes an instruction in one clock cycle**
  - This means that **no datapath resource can be used more than once** per instruction
- **Elements needed more than once must be duplicated**
  - Hence, we need **separate instruction and data memories**
  - A **dedicated adder for branch target address calculation**, separate from the ALU
- **Many of the elements can be shared by different instruction flows**
  - Use **multiplexers** and **control signals** where alternate data sources are used for different instructions

Multiplexers



- **Can't just join wires together**
  - Use multiplexers

A Simple Datapath for RISC-V



Example Program

- **For the sake of concreteness, we will have the single-cycle processor run this short program**
  - (We will only detail the first instruction. Refer to the book for the complete example)

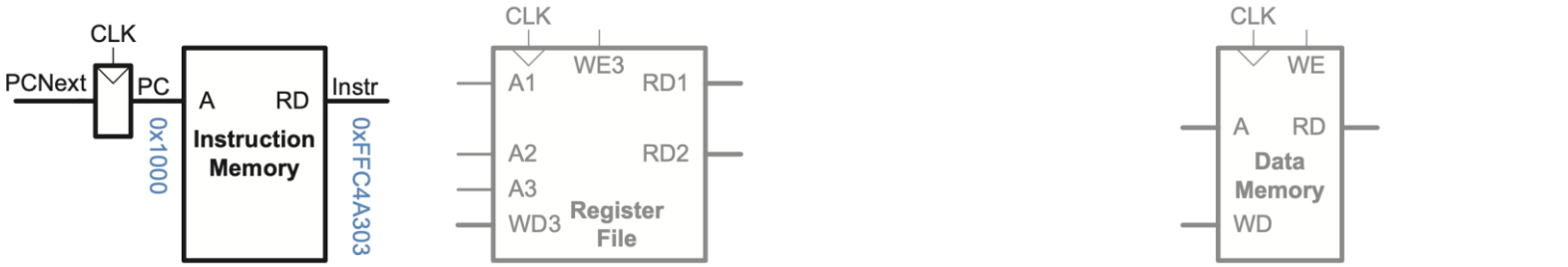
Address	Instruction	Type	Fields						Machine Language
			$imm_{11:0}$		rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100		01001	010	00110	0000011	FFC4A303
			$imm_{11:5}$	rs2	rs1	f3	$imm_{4:0}$	op	
0x1004	sw x6, 8(x9)	S	0000000	00110	01001	010	01000	0100011	0064A423
			funct7	rs2	rs1	f3	rd	op	
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233
			$imm_{12,10:5}$	rs2	rs1	f3	$imm_{4:1,11}$	op	
0x100C	beq x4, x4, L7	B	1111111	00100	00100	000	10101	1100011	FE420AE3

- **Assume that:**
  - `x9` contains the value 0x2004
  - Memory location 0x2000 contains the value 10

Single-Cycle Datapath

- **Although everything occurs in one clock cycle, we can think of four steps to execute the `lw` instruction**

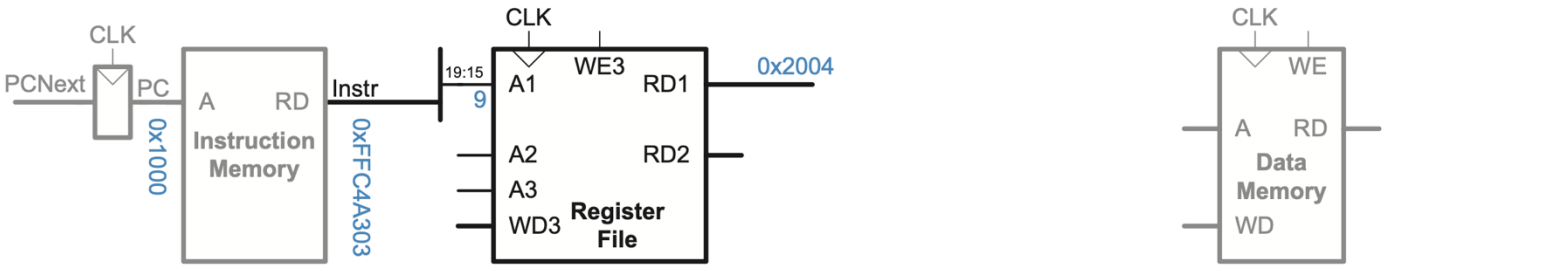
- **Step 1: Fetch instruction from memory**
  - The processor's actions depend on the specific instruction that was fetched



Address	Instruction	Type	Fields				Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Single-Cycle Datapath

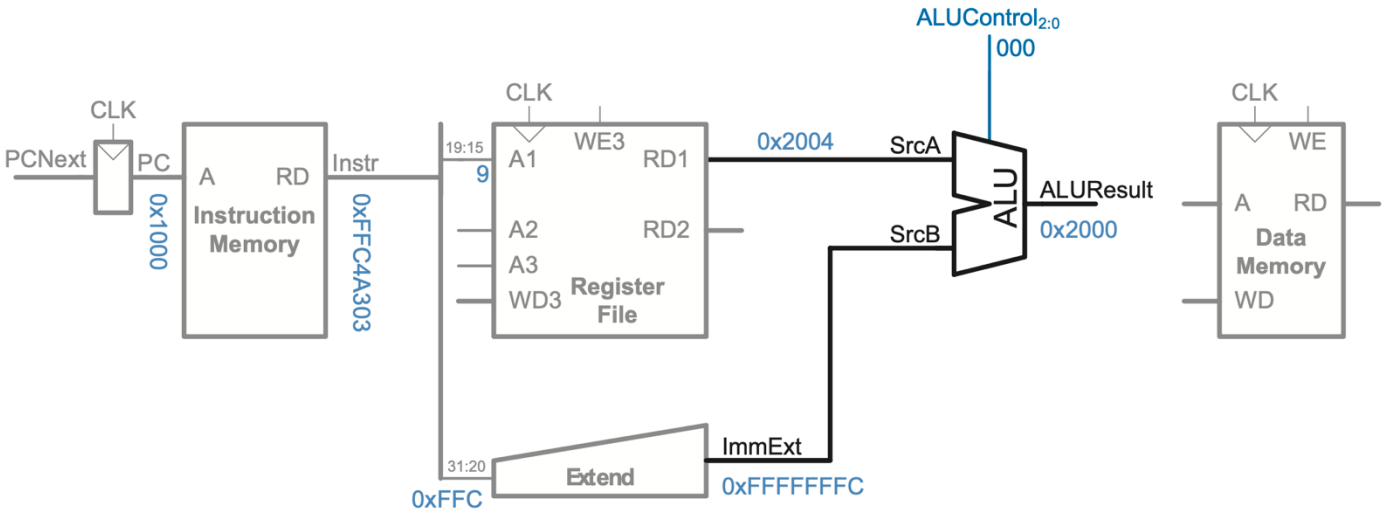
- **Step 2: Read source operand from register file**
  - A register (x9) value is read from the register file



Address	Instruction	Type	Fields					Machine Language
			$\mathbf{imm_{11:0}}$	**rs1**	**f3**	**rd**	**op**	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Single-Cycle Datapath

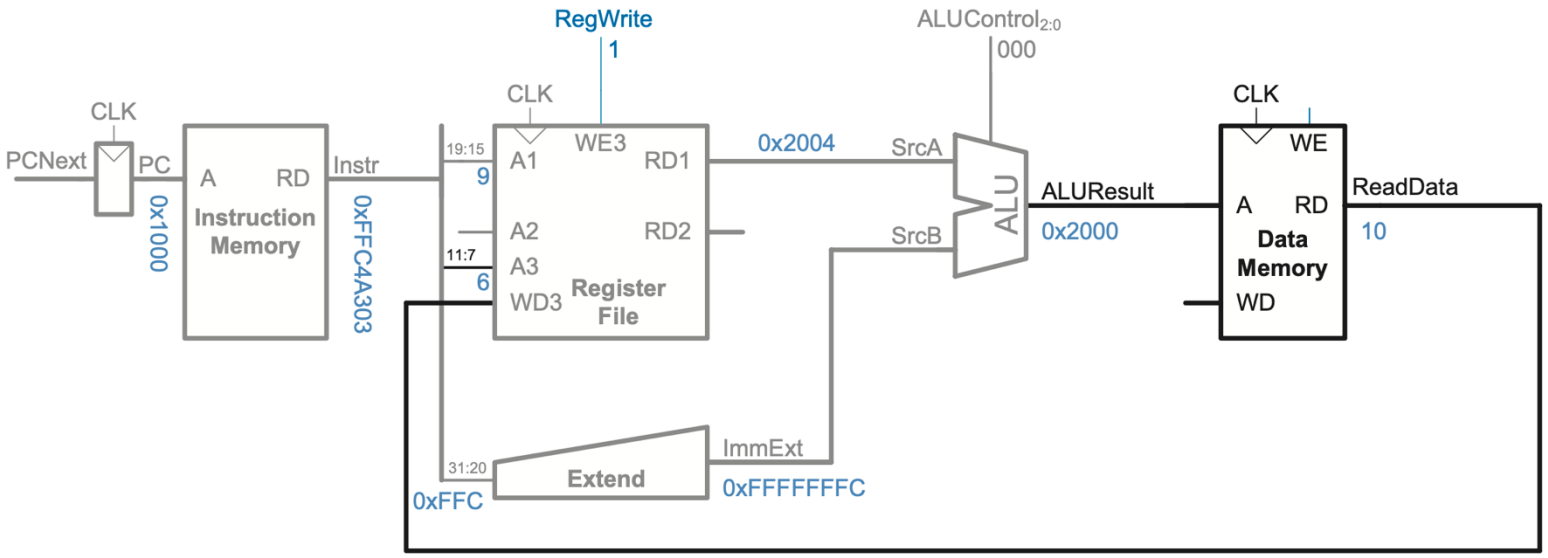
- **Step 3: Compute memory address**
  - The ALU computes the sum of the value **read from the register file** and the **sign-extended 12-bit immediate** of the instruction



Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Single-Cycle Datapath

- **Step 4: Read memory and write result back to register file**
  - The sum from the ALU is used as the address for the data memory



Address	Instruction	Type	Fields					Machine Language
			$imm_{11:0}$	rs1	f3	rd	op	
0x1000	L7: lw x6, -4(x9)	I	111111111100	01001	010	00110	0000011	FFC4A303

Checkpoint

- **This simple datapath combines the elements required by different instruction classes**
  - `lw`/`sw`, ALU operations, and branches
- **It can execute these basic instructions in a single clock cycle**
- **In the next slides, we refine this view to fill in the details, which requires that we:**
  1. Add **further functional units**
  2. Increase the **number of connections** between units
  3. Add a **control unit** to control what actions are taken for different instruction classes

The Control Unit

- **Control signals determine which specific instruction is performed by the datapath at any given time**

- **The control unit must ensure that:**
  1. The ALU performs the required operation
  2. The correct registers are read and, when needed, written
  3. Data memory is read or written when required
  4. The correct value is written back to the register file
  5. The PC selects `PC + 4` or the branch target as appropriate

Design Principles

- **Any time a datapath element has an input that changes its behavior, it requires a control signal**
  - E.g., ALU operation, read/write

- **Any time you need to pass a different input based on the instruction, add a multiplexer with a control signal as the selector**
  - E.g., next PC, ALU input, register to write to

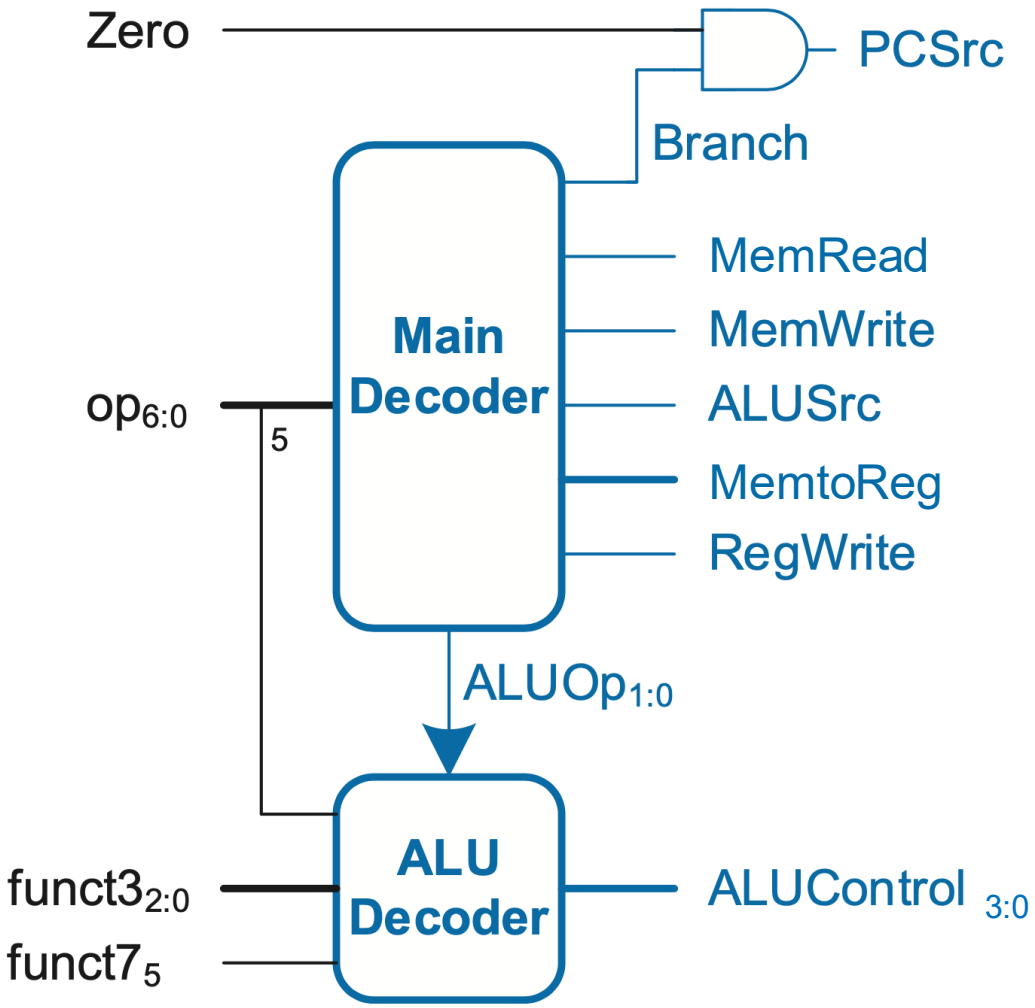
The Control Unit

- **The control unit must be able to:**
  - Take inputs from the instruction to be executed
  - Generate **write signals** for writable state elements, **selector controls** for multiplexers, and the **ALU control signals**

- **Using multiple levels of decoding is a common implementation technique**
  - The main control unit generates the `ALUOp` bits
  - Which are then used as input to the ALU control that generates the actual signals to control the ALU unit

- **This reduces the size and latency of the control unit**
  - Such optimizations are important, since the latency of the control unit **is often a critical factor** in determining the clock cycle time

The Control Unit



Designing the Main Decoder

- **Let's identify the fields of an instruction and the control lines that are needed for the datapath we constructed**

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) B-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Designing the Main Decoder

- **There are five major observations about these instruction formats that we will rely on:**
  1. The `opcode` field is **always in bits 6:0**
     - Depending on the opcode, the `funct3` field (bits 14:12) and `funct7` field (bits 31:25) serve as an extended opcode field
  2. Whenever present, `rs1` is **always in bit positions 19:15**
     - This field also specifies the base register for load and store instructions
  3. Whenever present, `rs2` is **always in bit positions 24:20**
     - This field also specifies the register operand that gets copied to memory for store instructions

Designing the Main Decoder

- **There are five major observations about these instruction formats that we will rely on:**
  4. Another operand can **also be a 12-bit offset** for branch or load-store instructions
  5. The destination register is always in **bit positions 11:7** (`rd`) for R-type instructions and load instructions

- **Key takeaway**
  - The first design principle – simplicity favours regularity – pays off here, **simplifying control of the datapath**

The Main Decoder 1-bit Outputs

- **Two 1-bit signals that are used to control multiplexors**
  - **ALUSrc** and **MemtoReg**
- **Three 1-bit signals for controlling reads and writes in the register file and data memory**
  - **RegWrite**, **MemRead**, and **MemWrite**
- **A 1-bit signal used in determining whether to possibly branch**
  - An AND gate is used to control the selection of the next PC
  - Combining the branch control signal (**Branch**) and the **Zero** output from the ALU

1-bit Control Signals

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

The ALU Decoder

- **Depending on the instruction class, the ALU will need to perform one of these three functions:**
  1. For load and store instructions, we use the ALU to compute the memory address by **addition**
  2. For the R-type instructions, the ALU needs to perform **one of the four actions** (`and`, `or`, `add`, or `sub`), depending on the `funct7` and `funct3` fields
  3. For the conditional branch if equal instruction, the ALU **subtracts** two operands and tests to see if the result is 0

- **So, the 2-bit control field ALUOp indicates whether the operation to be performed should be:**
  - `add` (00) for `lw`/`sw`, `sub` (01) for `beq`, or determined by the operation encoded in the `funct3` and `funct7` fields (10)

The Main Decoder

- **The logic for the main decoder can be developed from the following truth table**
  - It determines the instruction type from the `opcode` and then produces the appropriate control signals for the datapath
  - Each control signal can be 0, 1, or don't care (X)

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

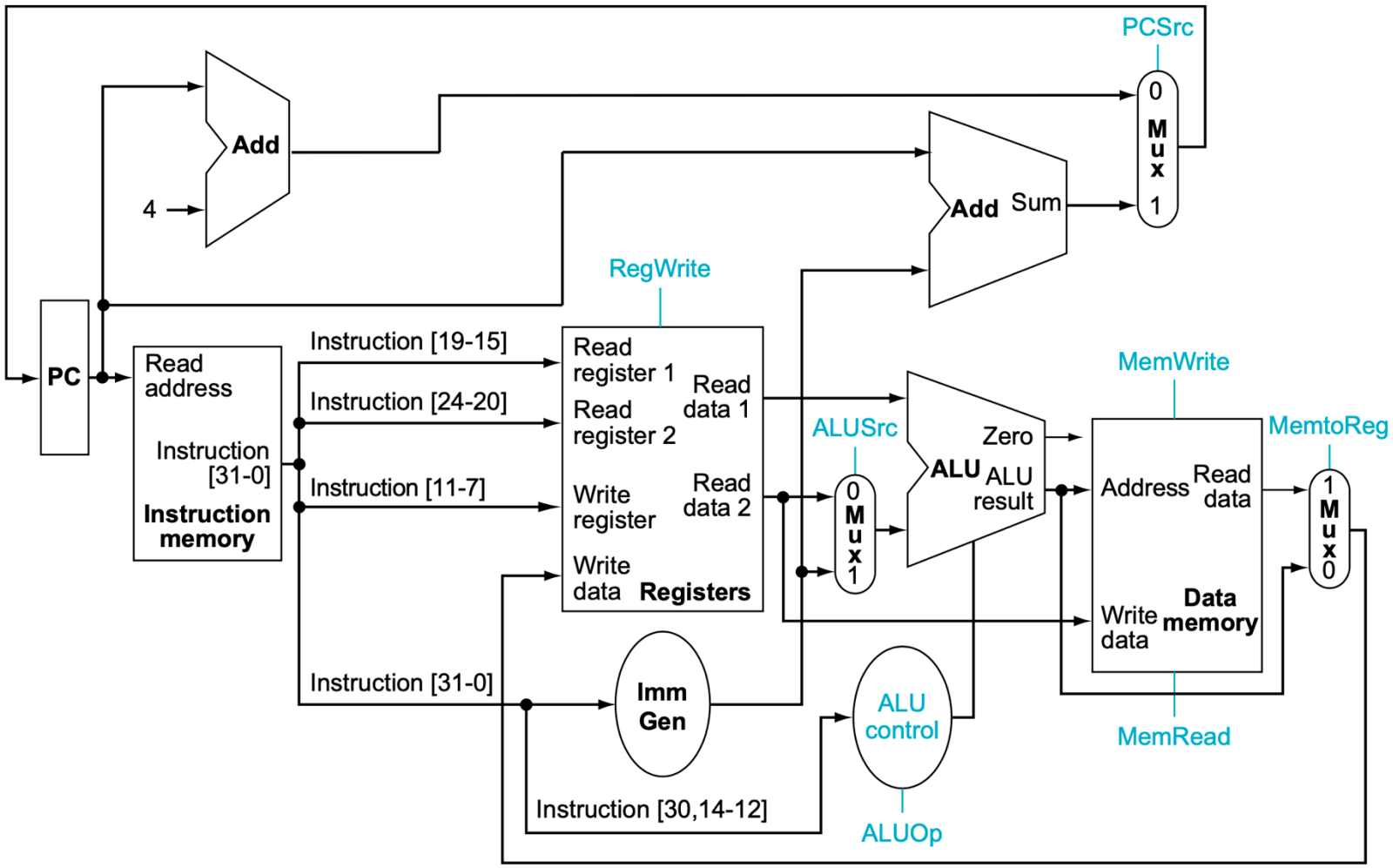
- **The PC does not require a write control, since it is written once at the end of every clock cycle**

The ALU Control

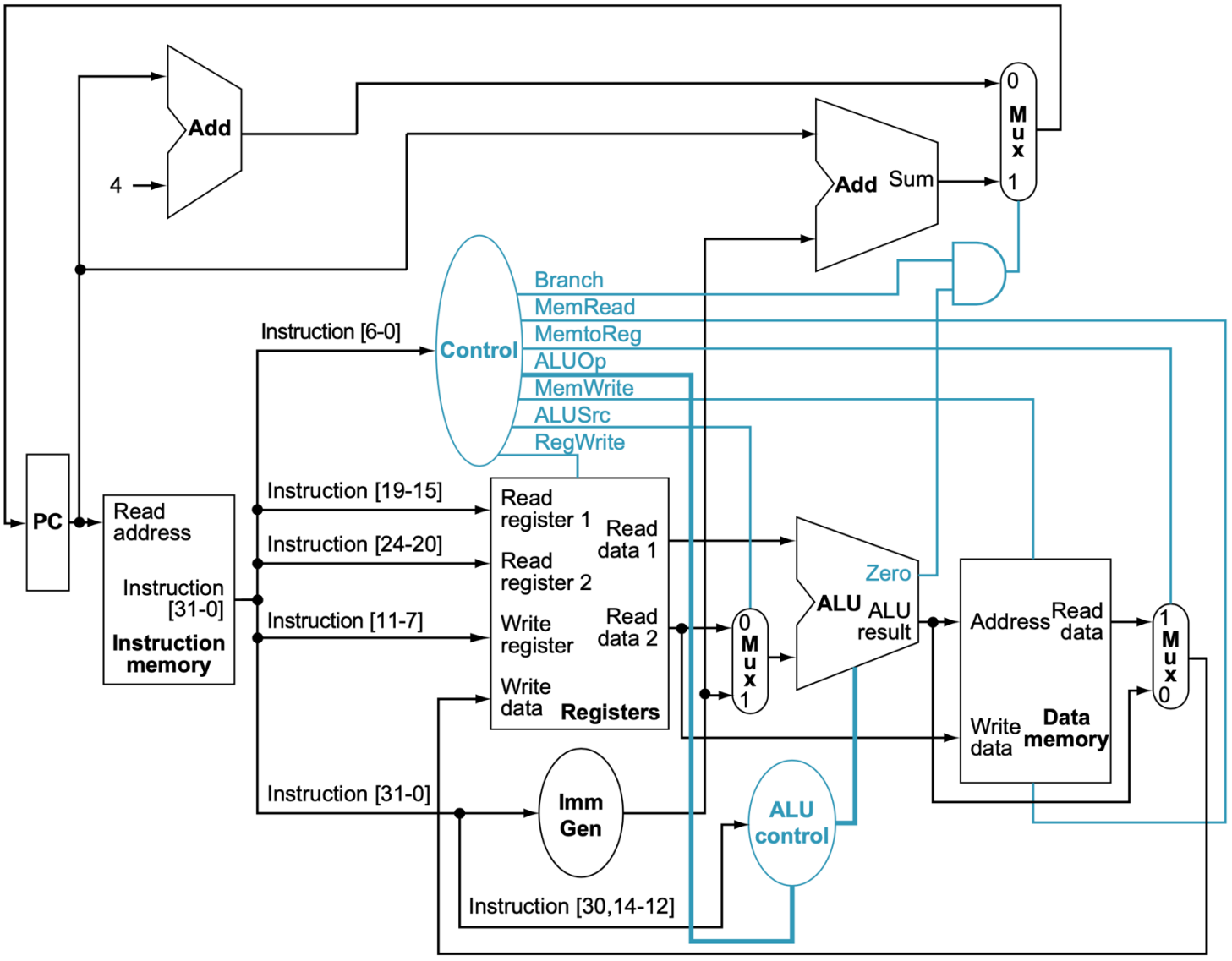
- **The 4-bit ALU control is derived from two inputs:**
  1. The `funct3` and part of the `funct7` fields of the instruction
  2. The 2-bit control field (**ALUOp**)

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

The ALU Control



Datapath With Control



Takeaway

- **What operations are involved when an instruction executes?**
  1. Fetch instruction from memory
  2. Decode the instruction and generate control signals
  3. Read registers
  4. Execute operation
  5. Access memory (if needed)
  6. Write result back
  7. Update PC

- **Datapath performs the work**

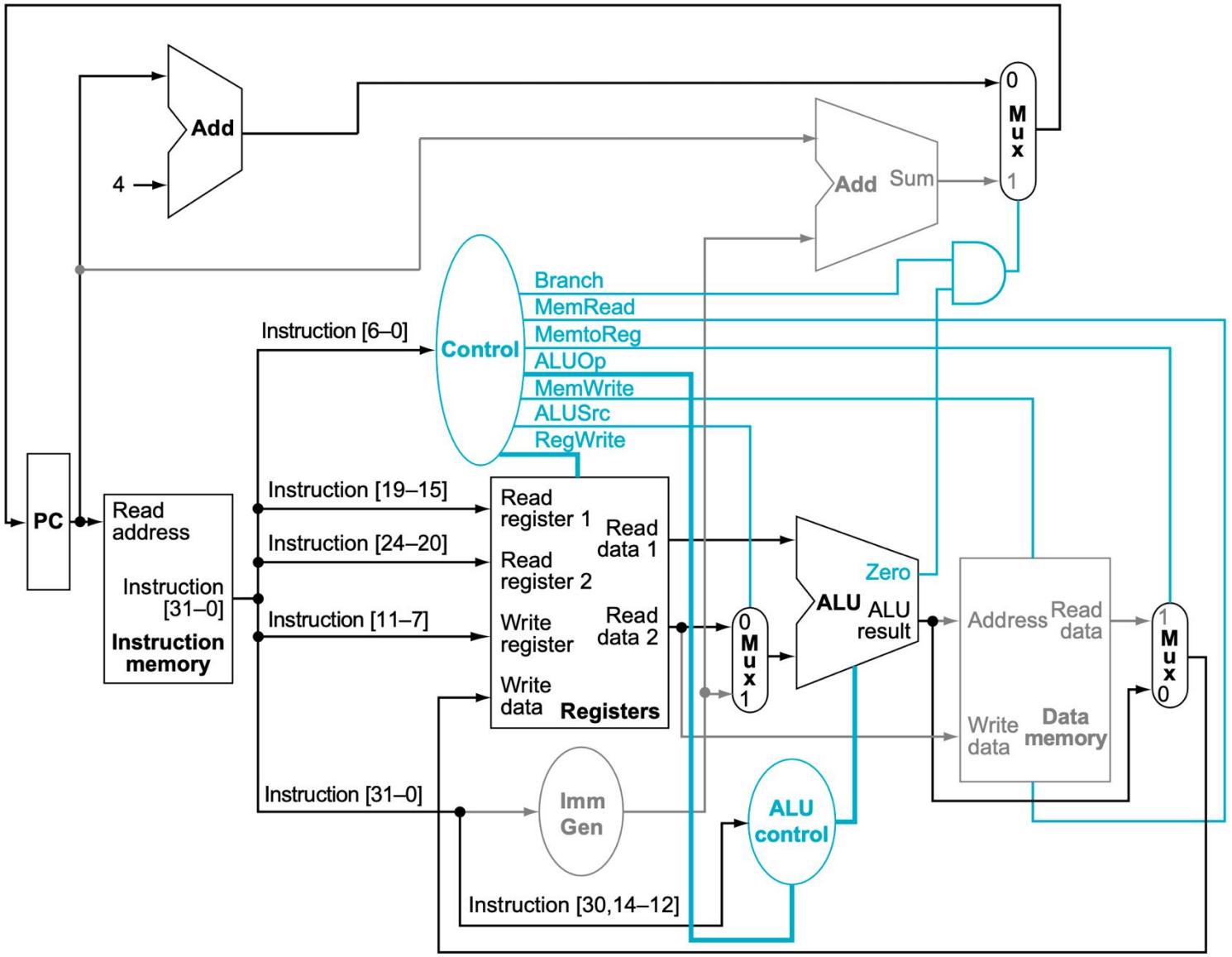
- **Control unit tells the datapath what to do**

Operation of the Datapath

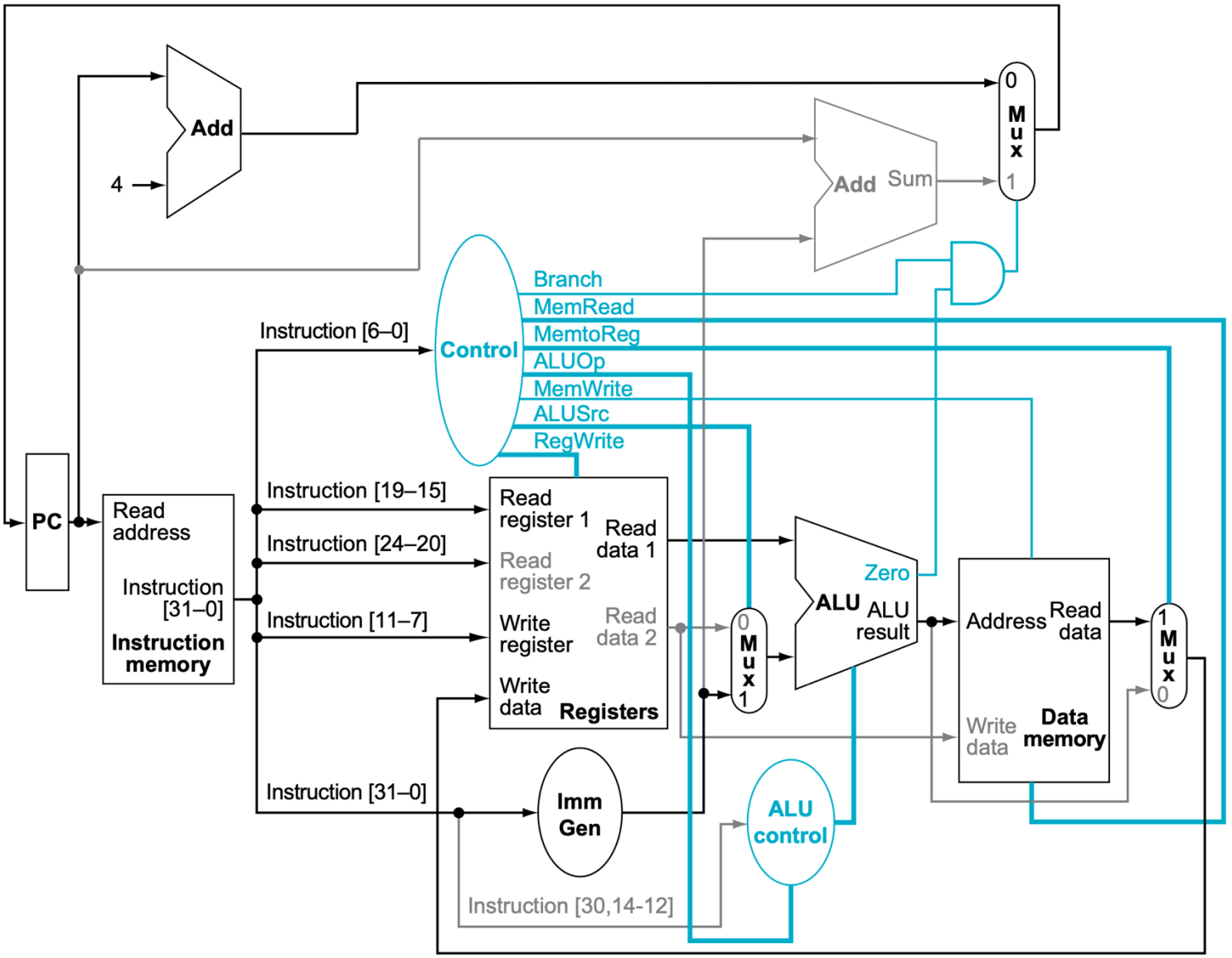
- **In the next few slides, we'll see the flow of three different instruction classes through the datapath**

- **The asserted control signals and active datapath elements are highlighted in each of these**
  - Note that a multiplexor whose control is 0 has a definite action, even if its control line is not highlighted
  - Multiple-bit control signals are highlighted if any constituent signal is asserted

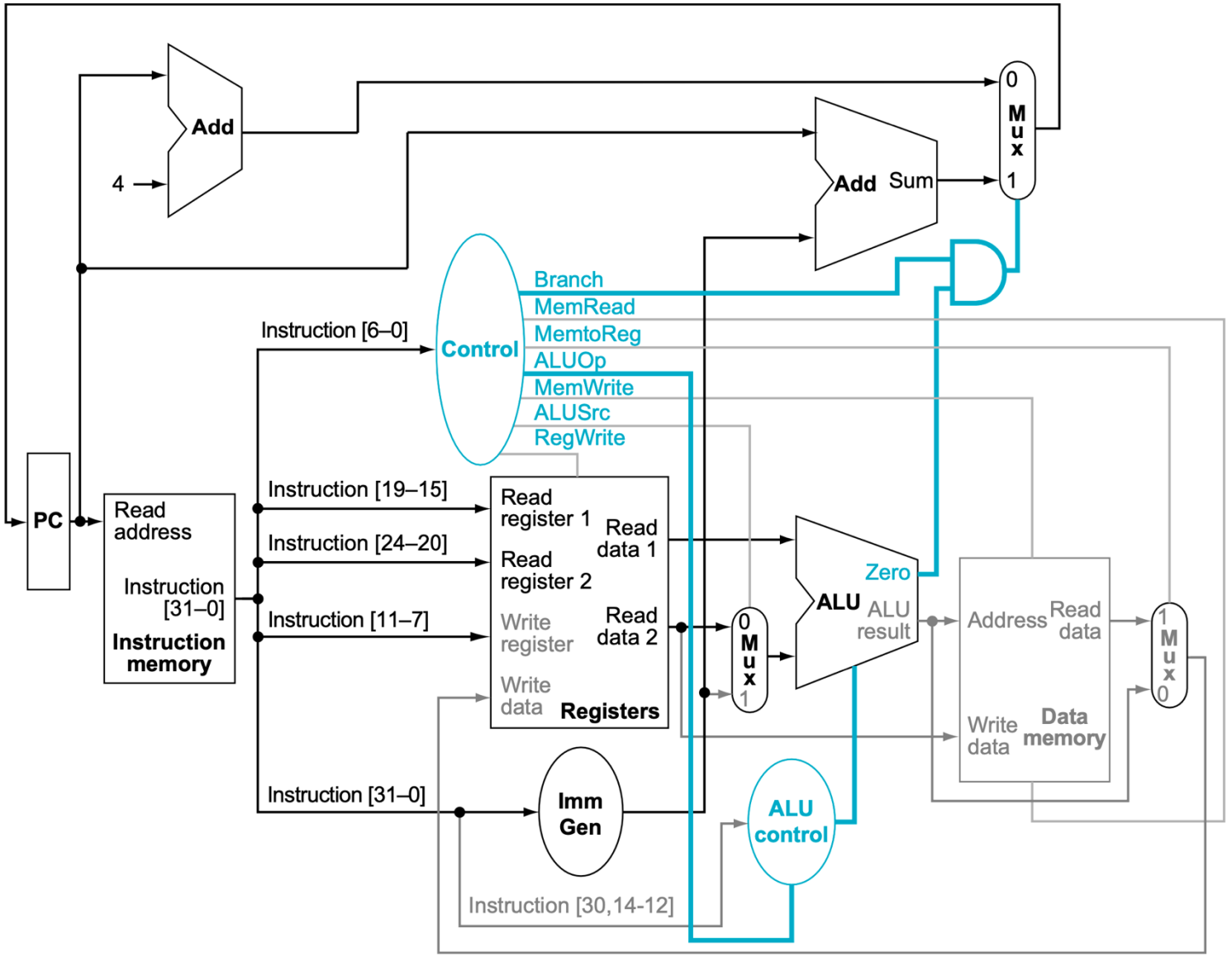
Example: `add x1, x2, x3`



Example: `lw x1, offset(x2)`



Example: `beq x1, x2, offset`



Performance Issues

- **Although the single-cycle design will work correctly, it is too inefficient to be used in modern designs**
- **All instructions execute in one clock cycle, so the CPI is 1**
- **The clock period must be long enough for the slowest instruction**
  - In our processor, the `lw` instruction
    - Instruction memory → register file → ALU → data memory → register file
  - Although all the other instructions (`sw`, `add`, `sub`, `and`, `or`, `beq`) take less time, they still execute in the needed time for `lw`

Performance Issues

- **Not feasible to vary the clock period for different instructions**
  - Overhead can easily surpass benefits due to difficulty in implementation
- **Violates the "making the common case fast" design principle**
- **We will improve performance by pipelining**
  - Uses a datapath very similar to the single-cycle datapath but is much more efficient by having a much **higher throughput**
  - Pipelining improves efficiency by executing multiple instructions simultaneously

Concluding Remarks

- **A datapath contains all the functional units and connections necessary to implement an instruction set architecture**
  - For our single-cycle implementation, we use two separate memories, an ALU, some extra adders, and some multiplexers
- **The control unit tells the datapath what to do, based on the instruction that's currently being executed**
  - Our processor has eight control signals that regulate the datapath
  - The control signals can be generated by a combinational circuit with the instruction's 32-bit binary encoding as input

Concluding Remarks

- **ISA influences design of datapath and control**
  - The datapath and control for a processor can be designed starting with the instruction set architecture
  - The simplicity and regularity of the RISC-V instruction set simplifies the implementation by making the execution of many of the instruction classes similar

- **Datapath and control influence design of ISA**
  - The underlying technology also affects many design decisions by dictating what components can be used in the datapath, as well as whether a single-cycle implementation even makes sense